



Stream API编程基础

北京理工大学计算机学院
金旭亮

概述



Stream API中定义了诸多的操作，能完成各种常见的数据处理工作。



了解并熟悉这些基本操作，需要花费一些时间，有些操作理解起来还会比较费劲，但这个时间是必须要花的，否则，你很难在开发中用好它们。

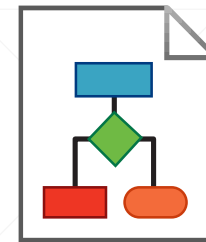


在后面的内容中，我们将通过大量的示例代码，展示这些流处理操作函数的基本使用方法。



强调一下：掌握这些基本操作，要求扎实掌握Java 8的Lambda表达式特性，并且熟悉JDK中相关的函数式接口。

基本编程套路



```
集合对象.stream()  
        .filter(...)  
        ...  
        .collect(...)
```

创建Stream对象

若干个中间操作

一个终结操作



对集合对象调用`stream()`方法，获取Stream对象的引用，之后，可以级联调用多个Stream API函数，最后，需要调用一个“终结操作”。

牢记!



所有的Stream API函数，都可以归结为以下两个类别：

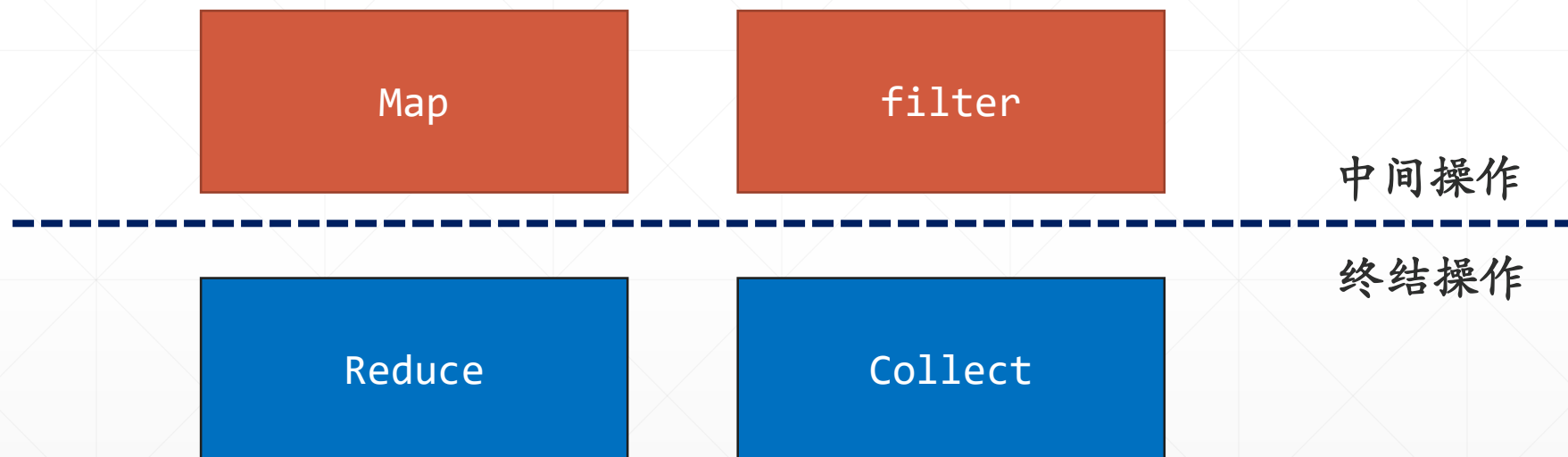
(1) 中间操作

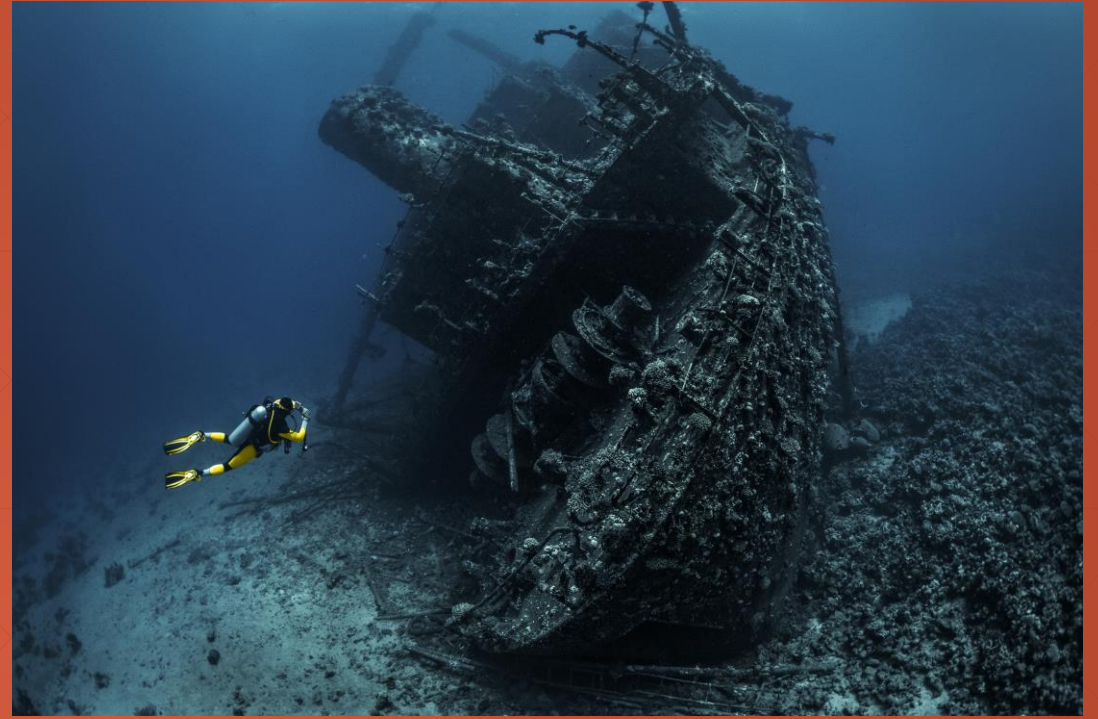
(2) 终结操作

在学习各个Stream API函数的过程中，首要的就是先弄明白这个函数属于哪个类型，之后才是了解它所实现的功能，最后通过编写一些Demo来掌握它的用法。

Stream API中的流操作

主要分为以下四类：





编程访问流中的数据

使用forEach遍历流中的元素



forEach()用于遍历一个集合，依次取出每个元素，然后给它施加一个特定处理（使用Lambda表达式表示），其定义如下：

```
void forEach(Consumer<? super T> action);
```

示例：

```
public static void useForEach() {  
    var people : List<Person> = Person.persons();  
    System.out.println("涨工资前: ");  
    people.stream().forEach(System.out::println);  
    System.out.println();  
    System.out.println("涨10%工资后: ");  
    people.stream().forEach(p -> p.setIncome(p.getIncome() * 1.1));  
    people.stream().forEach(System.out::println);  
}
```

VisitStreamDemo.java

思索

```
public static void useForEach() {  
    var people : List<Person> = Person.persons();  
    System.out.println("涨工资前: ");  
    people.stream().forEach(System.out::println);  
    System.out.println();  
    System.out.println("涨10%工资后: ");  
    people.stream().forEach(p -> p.setIncome(p.getIncome() * 1.1));  
    people.stream().forEach(System.out::println);  
}
```



上述示例中的forEach()方法，是针对Stream()调用的，事实上，people集合（类型为List<Person>）也有一个forEach()方法，功能一样，这两个方法是一回事吗？请通过查看其源码解决这个问题：

people.**forEach**(System.out::println)

使用peek“检查”流中的元素

peek的功能与forEach()非常类似，不同之处在于peek是一个中间操作，可以级联其他Stream API操作，而forEach则是一个终结操作。

```
Stream<T> peek(Consumer<? super T> action);
```

//使用peek加工每个元素

```
static void usePeek() {  
    var people : List<Person> = Person.persons();  
    people.stream()  
        .peek(p -> p.setIncome(p.getIncome() * 1.1))  
        .forEach(System.out::println);  
}
```



在数据处理管线的中部，不要修改原始集合中数据，它违反了函数式编程“避免编写有副作用函数”的大原则。

limit() 和 skip()

定义:

```
Stream<T> limit(long maxSize);
```

```
public static void useLimit() {  
    var nums : List<Integer> = List.of(1,2,3,4,5,6);  
    //取出前3个元素  
    nums.stream().limit(3).forEach(System.out::println);  
}
```

定义:

```
Stream<T> skip(long n);
```

```
static void useSkip(){  
    var nums : List<Integer> = List.of(1,2,3,4,5,6);  
    //忽略前3个元素, 输出后面的元素  
    nums.stream().skip(3)  
        .forEach(System.out::println);  
}
```

练习



`skip()`和`limit()`常用于对大数据集进行“分页”提取。

请写一个方法，这个方法能将一个大的数据集进行“分页”，然后，应外界请求提取相应的页返回。